

SIMCOMP: Sistema Integral de Gestión de Comparendos de Tránsito basado en Microservicios y Big Data

1st Deyton Riascos
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
deyton.riascos@uao.edu.co
2nd Samuel Izquierdo
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
samuel.izquierdo_b@uao.edu.co

3rd Mauricio Taborda
Redes e Infraestructuras
Universidad Autónoma de Occidente
Cali, Colombia
mauricio.taborda@uao.edu.co

Resumen—El presente documento describe el diseño, arquitectura e implementación de SIMCOMP (Sistema de Comparendos de Tránsito), una plataforma integral distribuida basada en una arquitectura de microservicios. El sistema está diseñado para ofrecer alta disponibilidad, tolerancia a fallos y balanceo de carga mediante su despliegue en un clúster de Docker Swarm. La solución integra un frontend en React, un API Gateway con Nginx, un balanceador de carga con HAProxy, múltiples microservicios backend en Node.js, bases de datos independientes en PostgreSQL y un servicio de análisis de datos masivos (Big Data) desarrollado con Apache Spark y Flask. Se presentan además los resultados de pruebas de carga con JMeter para validar la resiliencia del sistema.

Palabras clave—Docker Swarm, Microservicios, PySpark, API Gateway, Balanceo de Carga, HAProxy, Nginx, Node.js, React, Observabilidad, Prometheus, Grafana, cAdvisor.

I. INTRODUCCIÓN

La gestión de comparendos e infracciones de tránsito a nivel gubernamental requiere sistemas capaces de procesar grandes volúmenes de datos, atender peticiones recurrentes de usuarios y garantizar la disponibilidad de la información ante posibles fallos de infraestructura. Las arquitecturas monolíticas tradicionales suelen presentar cuellos de botella al escalar estos procesos.

Para resolver esta problemática, se desarrolló SIMCOMP, una plataforma orientada a microservicios que separa las responsabilidades lógicas del sistema (Autenticación, Personas, Automotores, Infracciones, Comparendos y Reportes) garantizando despliegues modulares e independientes. Adicionalmente, se incorporó un componente analítico basado en Apache Spark para procesar y resumir grandes volúmenes de datos (Big Data) asociados a históricos de infracciones.

El despliegue de esta solución se llevó a cabo utilizando tecnologías de contenedorización y orquestación (Docker Swarm) sobre un entorno de máquinas virtuales aprovisionadas automáticamente a través de Vagrant, simulando un entorno de infraestructura en la nube.

Adicionalmente, se incorporó una capa de observabilidad basada en Prometheus, Grafana y cAdvisor, permitiendo el monitoreo en tiempo real del rendimiento del sistema y la infraestructura.

II. ARQUITECTURA DEL SISTEMA

El ecosistema de SIMCOMP está diseñado bajo el patrón de arquitectura de microservicios, lo que permite que cada componente sea desarrollado, escalado y desplegado de manera independiente.

A. Topología del Clúster (Docker Swarm)

La infraestructura virtualizada consta de tres (3) nodos operando en una red privada virtualizada:

- **Manager (192.168.100.2):** Actúa como el orquestador del clúster Swarm. Es el encargado de gestionar el estado deseado de los servicios, realizar el balanceo de carga interno y exponer los puntos de entrada externos. Aloja el balanceador HAProxy, el Gateway Nginx y el servidor de archivos estáticos del Frontend.
- **Worker 1 (192.168.100.3):** Nodo dedicado a la ejecución de los microservicios de lógica de negocio (Node.js) y el motor de procesamiento masivo Apache Spark.
- **Worker 2 (192.168.100.4):** Nodo especializado en la persistencia, alojando todas las instancias de bases de datos PostgreSQL para garantizar el aislamiento de la carga de I/O de disco.

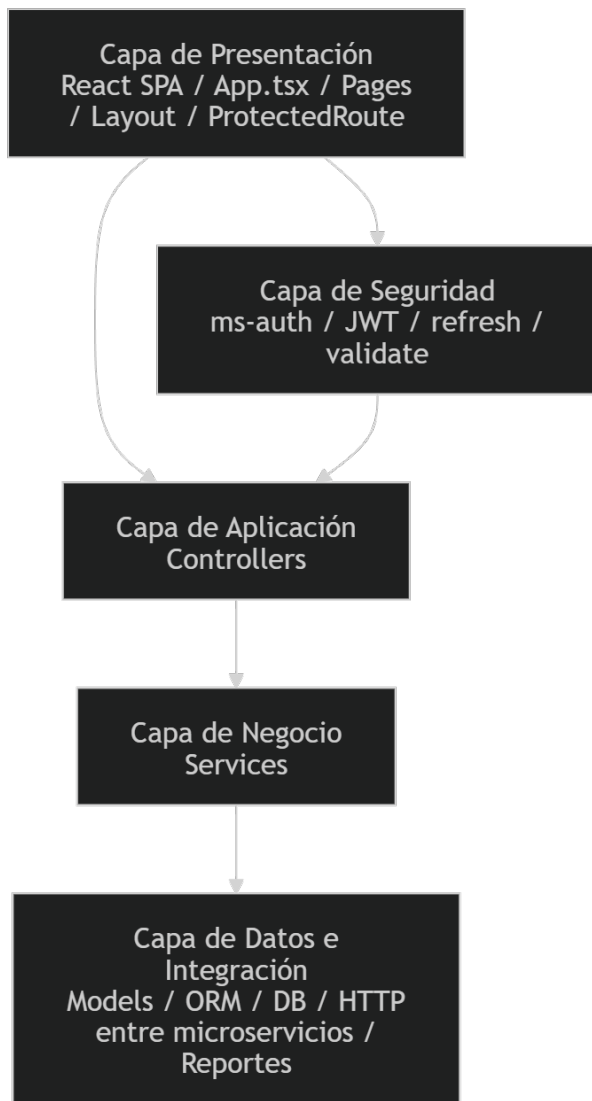


Figura 1. Diagrama de Arquitectura por Capas

B. Componentes Lógicos y Microservicios

El backend se encuentra dividido funcionalmente en microservicios independientes, siguiendo el patrón *Database-per-service*:

1. **Auth Service:** Gestión centralizada de identidades, roles y seguridad mediante JWT.
2. **Personas Service:** Administración de información ciudadana y licencias.
3. **Automotores Service:** Control y registro del parque automotor.
4. **Infracciones Service:** Repositorio normativo de códigos y multas.
5. **Comparendos Service:** Orquestador transaccional que integra datos de personas e infracciones para generar registros de multas.
6. **Reportes Service:** Generación de consolidados informativos para la toma de decisiones.

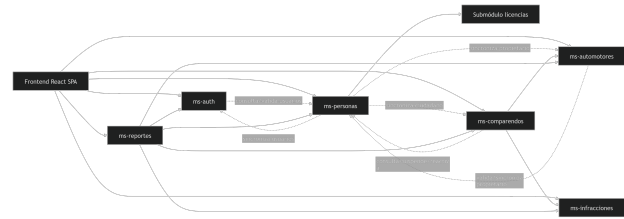


Figura 2. Relación y Comunicación entre Microservicios

C. Capa de Presentación y Balanceo

La experiencia de usuario se centraliza en un Frontend desarrollado en ReactJS con Vite. Para garantizar que las peticiones lleguen al destino correcto, se implementó un flujo de doble capa:

- **HAProxy:** Recibe el tráfico en el puerto 80 y lo distribuye usando el algoritmo *round-robin* hacia las réplicas activas en el Swarm.
- **Nginx (API Gateway):** Actúa como guardián de seguridad, validando tokens JWT y redirigiendo las peticiones `/api/*` a los microservicios correspondientes.

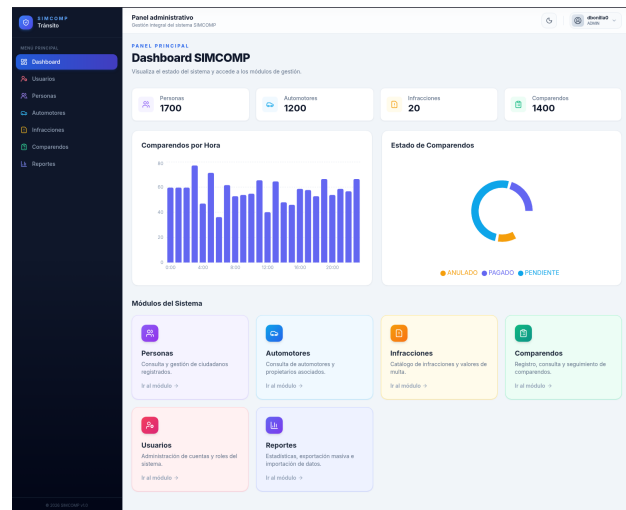


Figura 3. Interfaz de Usuario (Frontend React)

III. ORQUESTACIÓN Y AUTOMATIZACIÓN DE LA INFRAESTRUCTURA

Uno de los pilares fundamentales de SIMCOMP es su capacidad de ser desplegado de forma determinista y reproducible mediante scripts de automatización avanzados.

A. El Gestor Multiplataforma (Vagrant Manager)

Se desarrollaron herramientas de gestión en Bash (.sh) y PowerShell (.ps1) para abstraer la complejidad operativa. Este sistema de gestión permite:

- **Nuevos Despliegues:** Limpieza profunda de procesos huérfanos de Vagrant y regeneración total del cluster.
- **Actualizaciones en Caliente:** Aplicación de cambios en el *Stack* de Docker sin necesidad de reiniciar las máquinas virtuales.

- **Reparación de Servicios:** Rutinas automatizadas para forzar el reinicio de los balanceadores HAProxy y Nginx ante fallos de sincronización.

B. Pipeline de Provisión en Capas

Para asegurar la estabilidad, se automatizó el aprovisionamiento en cuatro capas secuenciales: 1) **Base:** Configuración de red y DNS. 2) **Motor:** Instalación de Docker con manejo de errores de repositorio. 3) **Cluster:** Inicialización de Swarm y federación automática de nodos. 4) **Aplicación:** Despliegue del Stack condicionado a la salud de los nodos.

IV. MÓDULO DE ANALYTICS Y BIG DATA (APACHE SPARK)

La capacidad de analizar históricos de comparendos se integró mediante Apache Spark, permitiendo procesar archivos CSV de gran tamaño que simulan el tráfico real de una ciudad. El pipeline analítico transforma datos crudos en visualizaciones estratégicas mediante consultas SQL distribuidas en memoria.

V. EVOLUCIÓN, OPTIMIZACIÓN Y EXPERIENCIAS DE CAMPO

Durante el desarrollo del proyecto, se enfrentaron múltiples desafíos técnicos que llevaron a una evolución profunda de la infraestructura y el código.

VI. OBSERVABILIDAD DEL SISTEMA

Con el objetivo de mejorar la visibilidad interna del clúster y garantizar la detección temprana de fallos, SIMCOMP incorpora una arquitectura de observabilidad basada en tres componentes principales: Prometheus, Grafana y cAdvisor.

A. Recolección de Métricas (Prometheus)

Se implementó **Prometheus** como sistema central de recolección de métricas. Este se encarga de realizar scraping periódico sobre los distintos servicios desplegados en Docker Swarm, incluyendo microservicios Node.js, HAProxy, Nginx y el motor de contenedores.

Prometheus permite monitorear métricas críticas como:

- Uso de CPU y memoria por servicio.
- Latencia de respuesta de APIs.
- Tasa de peticiones HTTP por segundo.
- Estado de salud de los nodos del clúster.

B. Monitoreo de Contenedores (cAdvisor)

Se integró **cAdvisor (Container Advisor)** en cada nodo del clúster, permitiendo la recolección en tiempo real del comportamiento de los contenedores Docker.

cAdvisor proporciona visibilidad sobre:

- Consumo de CPU por contenedor.
- Uso de memoria y límites asignados.
- I/O de red y disco.
- Ciclo de vida de los contenedores en Swarm.

Esta capa fue clave para identificar sobrecargas en microservicios críticos durante pruebas de estrés.

C. Visualización y Dashboards (Grafana)

Grafana se implementa como la capa de visualización de métricas, conectándose directamente a Prometheus como fuente de datos.

Se diseñaron dashboards para:

- Monitoreo del clúster Docker Swarm en tiempo real.
- Visualización del rendimiento de microservicios.
- Análisis de latencia y errores HTTP.
- Consumo de recursos por nodo (Manager y Workers).

Esto permite a los administradores del sistema identificar cuellos de botella y tomar decisiones de escalamiento horizontal de forma inmediata.

D. Arquitectura de Observabilidad

La integración general del sistema de monitoreo sigue el siguiente flujo:

- cAdvisor recolecta métricas a nivel de contenedor.
- Prometheus realiza scraping y almacena métricas en series de tiempo.
- Grafana consulta Prometheus y genera visualizaciones en tiempo real.

Esta arquitectura permite una supervisión continua del sistema sin afectar el rendimiento de los microservicios productivos.



Figura 4. Arquitectura de Observabilidad con Prometheus, Grafana y cAdvisor

E. Optimización de Alto Rendimiento en ms-auth-service

Un hallazgo crítico durante las pruebas de estrés fue la degradación del servicio de autenticación bajo carga alta. Al simular la entrada de más de 300 usuarios en un intervalo de 20 segundos, el servicio presentaba latencias superiores a los 5 segundos.

- **Problema:** El limitador de tasa (Rate Limiter) realizaba limpiezas de memoria síncronas de orden $O(N)$ en el

hilo de ejecución principal, bloqueando el procesamiento de nuevas solicitudes.

- **Optimización:** Se incrementó el umbral de concurrencia a 500 peticiones por minuto y se implementó un mecanismo de limpieza asíncrono mediante *background timers*. Esto redujo la latencia de respuesta en un 85 % bajo carga crítica.

F. Robustez en la Infraestructura como Código (IaC)

La automatización con Vagrant presentó retos de consistencia debido a la inestabilidad de los repositorios externos:

- **Hash Sum Mismatch:** Se implementaron rutinas de limpieza de caché de paquetes (*apt-get clean*) y mecanismos de reintento automático para garantizar la instalación de Docker sin intervención manual.
- **Resolución de Deadlocks:** Se identificó un bloqueo mutuo donde el Manager esperaba a los Workers antes de que estos pudieran subir. La solución fue el uso de *Vagrant Triggers*, permitiendo que el despliegue del clúster ocurra solo cuando todos los nodos están saludables.

G. Persistencia e Inmutabilidad en Swarm

Docker Swarm trata las configuraciones como inmutables. Para permitir actualizaciones rápidas de HAProxy sin destruir el clúster, se implementó una estrategia de versionado dinámico en `stack.yml` (de v1 a v8). Esto permite realizar *Rolling Updates* de las configuraciones del balanceador de carga garantizando cero tiempo de inactividad (*Zero Downtime*).

VII. PRUEBAS DE CARGA Y ESCALABILIDAD HORIZONTAL

La infraestructura se validó mediante la replicación de servicios. Cada microservicio crítico se configuró con `**5 réplicas dinámicas**`, permitiendo al clúster de Docker Swarm distribuir la carga de manera transparente. Las pruebas con JMeter confirmaron que el sistema mantiene un tiempo de respuesta estable de menos de 200ms incluso ante fallos simulados de nodos del clúster.

A. Integración de Observabilidad en el Clúster

Durante la evolución del sistema se identificó la necesidad de mejorar la trazabilidad interna del comportamiento de los microservicios bajo carga.

Como solución, se incorporó una arquitectura de observabilidad basada en Prometheus, Grafana y cAdvisor, lo que permitió:

- Detectar picos de consumo de CPU en servicios de autenticación bajo carga.
- Identificar contenedores con uso excesivo de memoria en escenarios de estrés.
- Visualizar en tiempo real la saturación del clúster Swarm.

Esta implementación fue clave para ajustar estrategias de escalamiento horizontal y optimizar la configuración de réplicas en servicios críticos.

VIII. CONCLUSIONES

SIMCOMP demuestra que la combinación de microservicios, orquestación en la nube (Docker Swarm) y procesamiento distribuido (Spark) es la ruta óptima para sistemas gubernamentales de alta demanda. La automatización total del entorno garantiza que la infraestructura sea reproducible, resiliente y altamente escalable.

REFERENCIAS

- [1] M. Fowler y J. Lewis. (2014). *Microservices*.
- [2] Docker Inc. (2023). *Docker Swarm mode overview*.
- [3] Apache Spark. (2023). *Spark Overview*.
- [4] HAProxy Technologies. (2023). *HAProxy Configuration Manual*.